Document number: P3524R0
Date: 2024-11-22
Project: Programming Language C++
Reference: ISO/IEC 14882:2024
Reply to: Jens Maurer
jens.maurer@gmx.net

Core Language Working Group "ready" Issues for the November, 2024 meeting

References in this document reflect the section and paragraph numbering of document WG21 N4993.

1953. Data races and common initial sequence

Section: 6.7.1 [intro.memory] **Status:** ready **Submitter:** Faisal Vali **Date:** 2014-06-23

According to 6.7.1 [intro.memory] paragraph 3,

> A *memory location* is either an object of scalar type or a maximal sequence of adjacent bit-fields all having non-zero width. [*Note:* Various features of the language, such as references and virtual functions, might involve additional memory locations that are not accessible to programs but are managed by the implementation. —*end note*] Two or more threads of execution (6.9.2 [intro.multithread]) can update and access separate memory locations without interfering with each other.

It is not clear how this relates to the permission granted in 11.4 [class.mem] paragraph 18 to inspect the common initial sequence of standard-layout structs that are members of a standard-layout union. If one thread is writing to the common initial sequence and another is reading from it via a different struct, that should constitute a data race, but the current wording does not clearly state that.

Additional notes (October, 2024)

(From submission #621.)

A similar concern arises for the following example:

```
union U { int x, y; } u;
(u.x = 1, 0) + (u.y = 2, 0);
```

Possible resolution [SUPERSEDED]:

Change in 6.7.1 [intro.memory] paragraph 3 as follows:

> A *memory location* is **the storage occupied by the object representation of** either an object of scalar type that is not a bit-field or a maximal sequence of adjacent bit-fields all having nonzero width. ...

CWG 2024-10-25

Subclause 6.9.1 [intro.execution] paragraph 10 does not cover unsequenced object creation that does not change any bits of storage, such as a placement new invoking a trivial default constructor. The original concern in this issue was addressed by P0137R1, adding the following in 11.4.2 [class.mfct] paragraph 28:

> In a standard-layout union with an active member (11.5 [class.union]) of struct type T1, it is permitted to read a non-static data member m of another union member of struct type T2 provided m is part of the common initial sequence of T1 and T2; the behavior is as if the corresponding member of T1 were nominated.

Proposed resolution (approved by CWG 2024-11-08):

1. Change in 6.7.1 [intro.memory] paragraph 3 as follows:

   > A *memory location* is **the storage occupied by the object representation of** either an object of scalar type that is not a bit-field or a maximal sequence of adjacent bit-fields all having nonzero width. ...

2. Change in 6.9.1 [intro.execution] paragraph 10 and add bullets as follows:

   > ... The value computations of the operands of an operator are sequenced before the value computation of the result of the operator. ~~If~~ **The behavior is undefined if**
   > - a side effect on a memory location (6.7.1 [intro.memory]) **or**
   > - starting or ending the lifetime of an object in a memory location
   >
   > is unsequenced relative to ~~either~~
   > - another side effect on the same memory location**,**
   > - **starting or ending the lifetime of an object occupying storage that overlaps with the memory location,** or
   > - a value computation using the value of any object in the same memory location,
   >
   > and ~~they~~ **the two evaluations** are not potentially concurrent (6.9.2 [intro.multithread]) ~~, the behavior is undefined~~. **[Note: Starting the lifetime of an object in a memory location can end the lifetime of objects in other memory locations (6.7.3 [basic.life]). -- end note]**
   >
   > [Note: ...]
   >
   > [Example:
   >
   > ```
   > void g(int i) {
   >   i = 7, i++, i++;      // i becomes 9
   > ```

```
    i = i++ + 1;    // the value of i is incremented
    i = i++ + i;    // undefined behavior
    i = i + 1;      // the value of i is incremented
    union U { int x, y; } u;
    (u.x = 1, 0) + (u.y = 2, 0);    // undefined behavior
  }
```

-- end example]

3. Change in 6.9.2.2 [intro.races] paragraph 2 as follows, adding bullets:

> Two expression evaluations *conflict* if one of them
> - modifies (3.1 [defns.access]) a memory location (6.7.1 [intro.memory]) **or**
> - starts or ends the lifetime of an object in a memory location
>
> and the other one
> - reads or modifies the same memory location **or**
> - **starts or ends the lifetime of an object occupying storage that overlaps with the memory location**.
>
> [*Note 2:* A modification can still conflict even if it does not alter the value of any bits. —*end note*]

1965. Explicit casts to reference types

Section: 7.6.1.7 [expr.dynamic.cast] **Status:** ready **Submitter:** Richard Smith **Date:** 2014-07-07

The specification of `dynamic_cast` in 7.6.1.7 [expr.dynamic.cast] paragraph 2 (and `const_cast` in 7.6.1.11 [expr.const.cast] is the same) says that the operand of a cast to an lvalue reference type must be an lvalue, so that

```
struct A { virtual ~A(); }; A &&make_a();

A &&a = dynamic_cast<A&&>(make_a());    // ok
const A &b = dynamic_cast<const A&>(make_a()); // ill-formed
```

The behavior of `static_cast` is an odd hybrid:

```
struct B : A { }; B &&make_b();
A &&c = static_cast<A&&>(make_b()); // ok
const A &d = static_cast<const A&>(make_b()); // ok
const B &e = static_cast<const B&>(make_a()); // ill-formed
```

(Binding a const lvalue reference to an rvalue is permitted by 7.6.1.9 [expr.static.cast] paragraph 4 but not by paragraphs 2 and 3.)

There is implementation divergence on the treatment of these examples.

Also, `const_cast` permits binding an rvalue reference to a class prvalue but not to any other kind of prvalue, which seems like an unnecessary restriction.

Finally, 7.6.1.9 [expr.static.cast] paragraph 3 allows binding an rvalue reference to a class or array prvalue, but not to other kinds of prvalues; those are covered in paragraph 4. This would be less confusing if paragraph 3 only dealt with binding rvalue references to glvalues and left all discussion of prvalues to paragraph 4, which adequately handles the class and array cases as well.

Notes from the May, 2015 meeting:

CWG reaffirmed the status quo for `dynamic_cast` but felt that `const_cast` should be changed to permit binding an rvalue reference to types that have associated memory (class and array types).

CWG 2024-11-19

Resolved by the resolution of issue 2879.

2283. Missing complete type requirements

Section: 7.6.1.3 [expr.call] **Status:** ready **Submitter:** Richard Smith **Date:** 2016-06-27

P0135R1 (Wording for guaranteed copy elision through simplified value categories) removes complete type requirements from 7.6.1.3 [expr.call] (under the assumption that subclause 9.4 [dcl.init] has them; apparently it does not) and from 7.6.1.8 [expr.typeid] paragraph 3. These both appear to be bad changes and should presumably be reverted.

Additional notes (October, 2024)

An almost-editorial change (approved by CWG 2021-08-24) restored a consistent complete-type requirement for `typeid`; see cplusplus/draft#4827.

Proposed resolution (approved by CWG 2024-10-25):

Change in 7.6.1.3 [expr.call] paragraph 13 as follows:

> A function call is an lvalue if the result type is an lvalue reference type or an rvalue reference to function type, an xvalue if the result type is an rvalue reference to object type, and a prvalue otherwise. **If it is a non-void prvalue, the type of the function call expression shall be complete, except as**

specified in 9.2.9.6 [dcl.type.decltype].

2815. Overload resolution for references/pointers to `noexcept` functions

Section: 12.2.4.3 [over.ics.rank] **Status:** ready **Submitter:** Brian Bi **Date:** 2023-10-05

Consider:

```
void f() noexcept {}

void g(void (*)() noexcept) {}
void g(void (&)()) {}

int main() {
  g(f);     // error: ambiguous
}
```

In contrast:

```
void f() noexcept {}

void g(void (*)()) {}
void g(void (&)()) {}      // #1

int main() {
  g(f);    // OK, calls #1
}
```

In both cases, binding `void(&)()` to `void() noexcept` is considered an identity conversion, without further disambiguation by 12.2.4.3 [over.ics.rank].

CWG 2024-06-26

Binding a reference to a function should not be considered an identity conversion if it strips a non-throwing exception specification. This amendment removes the ambiguity for the first example and makes the second example ambiguous, which is desirable.

Proposed resolution (approved by CWG 2024-10-11):

1. Change in 12.2.4.2.5 [over.ics.ref] paragraph 1 as follows:

> When a parameter of type "reference to cvT" binds directly (9.4.4 [dcl.init.ref]) to an argument expression:
> - If the argument expression has a type that is a derived class of the parameter type, the implicit conversion sequence is a derived-to-base conversion (12.2.4.2 [over.best.ics]).
> - Otherwise, ~~if T is a function type, or~~ if the type of the argument is possibly cv-qualified T, or if T is an array type of unknown bound with element type U and the argument has an array type of known bound whose element type is possibly cv-qualified U, the implicit conversion sequence is the identity conversion. ~~[*Note 1:* When T is a function type, the type of the argument can differ only by the presence of noexcept. —*end note*]~~
> - Otherwise, if T is a function type, the implicit conversion sequence is a function pointer conversion.
> - Otherwise, the implicit conversion sequence is a qualification conversion.
>
> [*Example 1:*
>
> ```
> struct A {};
> struct B : public A {} b;
> int f(A&);
> int f(B&);
> int i = f(b);     // calls f(B&), an exact match, rather than f(A&), a conversion
>
> void g() noexcept;
> int h(void (&)() noexcept); // #1
> int h(void (&)());          // #2
> int j = h(g);               // calls #1, an exact match, rather than #2, a function pointer conversion
> ```
>
> —*end example*]

2. Change in 12.2.4.3 [over.ics.rank] bullet 3.2.6 as follows:

```
int f(const int &);
int f(int &);
int g(const int &);
int g(int);
int i;
int j = f(i);   // calls f(int &)
int k = g(i);   // ambiguous

struct X {
  void f() const;
  void f();
};
void g(const X& a, X b) {
  a.f();     // calls X::f() const
  b.f();     // calls X::f()
}

int h1(int (&)[]);
```

```
  int h1(int (&)[1]);
  int h2(void (&)());
  int h2(void (&)() noexcept);
  void g2() {
    int a[1];
    h1(a);
    extern void f2() noexcept;
    h2(f2);
  }
```

2879. Undesired outcomes with `const_cast`

Section: 7.6.1.11 [expr.const.cast] **Status:** ready **Submitter:** Brian Bi **Date:** 2024-04-15

(From submissions #526, #232, and #342.)

The resolution of issue 891 intended to make `const_cast<int&&>(2)` ill-formed. However, combined with the temporary materialization conversion turning prvalues into glvalues (7.1 [expr.pre] paragraph 7, this is now well-formed.

Also, the current rules regrettably allow `const_cast<int>(0)` and const_casts involving function pointers and pointers to member functions. The latter is non-normatively considered disallowed by the note in 7.6.1.11 [expr.const.cast] paragraph 9.

Major implementations except MSVC agree with the proposed direction of this issue.

Suggested resolution [SUPERSEDED]:

1. Change in 7.6.1.11 [expr.const.cast] paragraph 3 as follows:

> ~~For two similar types T1 and T2 (7.3.6 [conv.qual]), a prvalue of type T1 may be explicitly converted to the type T2 using a const_cast if, considering the qualification-decompositions of both types, each Pi1 is the same as Pi2 for all i. The result of a const_cast refers to the original entity.~~
>
> If `T` is an object pointer type or pointer to data member type, the type of `v` shall be similar to `T` and the corresponding P_i components of the qualification decompositions of `T` and the type of `v` shall be the same (7.3.6 [conv.qual]). If `v` is a null pointer or null member pointer, the result is a null pointer or null member pointer, respectively. Otherwise, the result points to or past the end of the same object or member, respectively, as `v`.
>
> [Example: ...]

2. Change in 7.6.1.11 [expr.const.cast] paragraph 4 as follows:

> ~~For two object types T1 and T2, if a pointer to T1 can be explicitly converted to the type "pointer to T2" using a const_cast, then the following conversions can also be made:~~
>
> - ~~an lvalue of type T1 can be explicitly converted to an lvalue of type T2 using the cast const_cast<T2&>;~~
> - ~~a glvalue of type T1 can be explicitly converted to an xvalue of type T2 using the cast const_cast<T2&&>; and~~
> - ~~if T1 is a class type, a prvalue of type T1 can be explicitly converted to an xvalue of type T2 using the cast const_cast<T2&&>.~~
>
> ~~The result of a reference const_cast refers to the original object if the operand is a glvalue and to the result of applying the temporary materialization conversion (7.3.5 [conv.rval]) otherwise.~~
>
> Otherwise, T shall be a reference type. Let T1 be the type of v and T2 be the type referred to by T. A const_cast from "pointer to T1" to "pointer to T2" shall be valid. If T is an lvalue reference type, v shall be an lvalue. Otherwise, if T2 is a class type and v is a prvalue, the temporary materialization conversion (7.3.5 [conv.rval]) is applied. Otherwise, the temporary materialization conversion is not applied and v shall be a glvalue. The result refers to the same object as the (possibly converted) operand.

3. Remove 7.6.1.11 [expr.const.cast] paragraph 5:

> ~~A null pointer value (6.8.4 [basic.compound]) is converted to the null pointer value of the destination type. The null member pointer value (7.3.13 [conv.mem]) is converted to the null member pointer value of the destination type.~~

Suggested resolution [SUPERSEDED]:

1. Change in 7.6.1.11 [expr.const.cast] paragraph 1 as follows:

> The result of the expression const_cast<T>(v) is of type T. If T is an lvalue reference to object type, the result is an lvalue; if T is an rvalue reference to object type, the result is an xvalue; otherwise, the result is a prvalue and the lvalue-to-rvalue (7.3.2 [conv.lval]), array-to-pointer (7.3.3 [conv.array]), and function-to-pointer (7.3.4 [conv.func]) standard conversions are performed on the expression v. **The temporary materialization conversion (7.3.5 [conv.rval]) is not performed on v, other than as specified below.** Conversions that can be performed explicitly using const_cast are listed below. No other conversion shall be performed explicitly using const_cast.

2. Change in 7.6.1.11 [expr.const.cast] paragraph 3 as follows:

> For two similar **object pointer or pointer to data member** types T1 and T2 (7.3.6 [conv.qual]), a prvalue of type T1 ~~may~~ **can** be explicitly converted to the type T2 using a const_cast if, considering the qualification-decompositions of both types, each Pi1 is the same as Pi2 for all i. ~~The result of a const_cast refers to the original entity.~~ **If v is a null pointer or null member pointer, the result is a null pointer or null member pointer, respectively. Otherwise, the result points to or past the end of the same object, or points to the same member, respectively, as v.**

3. Change in 7.6.1.11 [expr.const.cast] paragraph 4 as follows:

For two object types T1 and T2, if a pointer to T1 can be explicitly converted to the type "pointer to T2" using a const_cast, then the following conversions can also be made:

- an lvalue of type T1 can be explicitly converted to an lvalue of type T2 using the cast const_cast<T2&>;
- a glvalue of type T1 can be explicitly converted to an xvalue of type T2 using the cast const_cast<T2&&>; and
- if T1 is a class type, a prvalue of type T1 can be explicitly converted to an xvalue of type T2 using the cast const_cast<T2&&>. **The temporary materialization conversion is performed on v**.

The result ~~of a reference const_cast~~ refers to the ~~original~~ **same** object ~~if~~ **as** the **(possibly converted)** operand ~~is a glvalue and to the result of applying the temporary materialization conversion (7.3.5 [conv.rval]) otherwise~~.

4. Remove 7.6.1.11 [expr.const.cast] paragraph 5:

~~A null pointer value (6.8.4 [basic.compound]) is converted to the null pointer value of the destination type. The null member pointer value (7.3.13 [conv.mem]) is converted to the null member pointer value of the destination type.~~

CWG 2024-05-31

The existing example in 7.6.1.11 [expr.const.cast] paragraph 3 shows the temporary materialization conversion applied to an array type. The example would be made ill-formed by the suggested resolution above. More investigation is advised.

Suggested resolution [SUPERSEDED]:

1. Change in 7.6.1.11 [expr.const.cast] paragraph 1 as follows:

The result of the expression const_cast<T>(v) is of type T. If T is an lvalue reference to object type, the result is an lvalue; if T is an rvalue reference to object type, the result is an xvalue; otherwise, the result is a prvalue and the lvalue-to-rvalue (7.3.2 [conv.lval]), array-to-pointer (7.3.3 [conv.array]), and function-to-pointer (7.3.4 [conv.func]) standard conversions are performed on the expression v. **The temporary materialization conversion (7.3.5 [conv.rval]) is not performed on v, other than as specified below.** Conversions that can be performed explicitly using const_cast are listed below. No other conversion shall be performed explicitly using const_cast.

2. Change in 7.6.1.11 [expr.const.cast] paragraph 3 as follows:

For two similar **object pointer or pointer to data member** types T1 and T2 (7.3.6 [conv.qual]), a prvalue of type T1 ~~may~~ **can** be explicitly converted to the type T2 using a const_cast if, considering the qualification-decompositions of both types, each Pi1 is the same as Pi2 for all i. ~~The result of a const_cast refers to the original entity.~~ **If v is a null pointer or null member pointer, the result is a null pointer or null member pointer, respectively. Otherwise, the result points to or past the end of the same object, or points to the same member, respectively, as v.** ~~[Example 1:~~

```
~~typedef int *A[3];                    // array of 3 pointer to int~~
~~typedef const int *const CA[3];      // array of 3 const pointer to const int~~
~~CA &&r = A{};            // OK, reference binds to temporary array object~~
                           ~~// after qualification conversion to type CA~~
~~A &&r1 = const_cast<A>(CA{});        // error: temporary array decayed to pointer~~

~~A &&r2 = const_cast<A&&>(CA{});      // OK~~
```

~~-- end example]~~

3. Change in 7.6.1.11 [expr.const.cast] paragraph 4 as follows:

For two object types T1 and T2, if a pointer to T1 can be explicitly converted to the type "pointer to T2" using a const_cast, then the following conversions can also be made:

- an lvalue of type T1 can be explicitly converted to an lvalue of type T2 using the cast const_cast<T2&>;
- a glvalue of type T1 can be explicitly converted to an xvalue of type T2 using the cast const_cast<T2&&>; and
- if T1 is a class **or array** type, a prvalue of type T1 can be explicitly converted to an xvalue of type T2 using the cast const_cast<T2&&>. **The temporary materialization conversion is performed on v**.

The result ~~of a reference const_cast~~ refers to the ~~original~~ **same** object ~~if~~ **as** the **(possibly converted)** operand ~~is a glvalue and to the result of applying the temporary materialization conversion (7.3.5 [conv.rval]) otherwise~~.

[Example 2:

```
typedef int *A[3];                  // array of 3 pointer to int
typedef const int *const CA[3];     // array of 3 const pointer to const int

auto &&r2 = const_cast<A&&>(CA{}); // OK, temporary materialization conversion is performed
```

-- end example]

4. Remove 7.6.1.11 [expr.const.cast] paragraph 5:

~~A null pointer value (6.8.4 [basic.compound]) is converted to the null pointer value of the destination type. The null member pointer value (7.3.13 [conv.mem]) is converted to the null member pointer value of the destination type.~~

5. Change in 9.4.4 [dcl.init.ref] bullet 5.3 as follows:

[Example 5: ...

```
constexpr int f() {
  const int &x = 42;
  const_cast<int &>(x) = 1;   // undefined behavior
  return x;
}
constexpr int z = f();            // error: not a constant expression
```

```
typedef int *AP[3];          // array of 3 pointer to int
typedef const int *const ACPC[3]; // array of 3 const pointer to const int
ACPC &&r = AP{};             // binds directly
```

-- end example]

This resolution also resolve issue 1965.

CWG 2024-10-11

Subclause 7.2.1 [basic.lval] paragraph 7 should be amended with "unless otherwise specified" and cross-references to the exceptions.

Proposed resolution (approved by CWG 2024-10-25)

1. Change in 7.2.1 [basic.lval] paragraph 7 as follows:

> ~~Whenever~~ **Unless otherwise specified (7.6.1.11 [expr.const.cast]), whenever** a prvalue appears as an operand of an operator that expects a glvalue for that operand, the temporary materialization conversion (7.3.5 [conv.rval]) is applied to convert the expression to an xvalue.

2. Change in 7.6.1.11 [expr.const.cast] paragraph 1 as follows:

> The result of the expression const_cast<T>(v) is of type T. If T is an lvalue reference to object type, the result is an lvalue; if T is an rvalue reference to object type, the result is an xvalue; otherwise, the result is a prvalue and the lvalue-to-rvalue (7.3.2 [conv.lval]), array-to-pointer (7.3.3 [conv.array]), and function-to-pointer (7.3.4 [conv.func]) standard conversions are performed on the expression v. **The temporary materialization conversion (7.3.5 [conv.rval]) is not performed on v, other than as specified below.** Conversions that can be performed explicitly using const_cast are listed below. No other conversion shall be performed explicitly using const_cast.

3. Change in 7.6.1.11 [expr.const.cast] paragraph 3 as follows:

> For two similar **object pointer or pointer to data member** types T1 and T2 (7.3.6 [conv.qual]), a prvalue of type T1 ~~may~~ **can** be explicitly converted to the type T2 using a const_cast if, considering the qualification-decompositions of both types, each Pi1 is the same as Pi2 for all i. ~~The result of a const_cast refers to the original entity.~~ **If v is a null pointer or null member pointer, the result is a null pointer or null member pointer, respectively. Otherwise, the result points to or past the end of the same object, or points to the same member, respectively, as v.** ~~[Example 1:~~
>
> ~~```~~
> ~~typedef int *A[3];   // array of 3 pointer to int~~
> ~~typedef const int *const CA[3];   // array of 3 const pointer to const int~~
> ~~CA &&r = A{};   // OK, reference binds to temporary array object~~
> ~~   // after qualification conversion to type CA~~
> ~~A &&r1 = const_cast<A>(CA{});   // error: temporary array decayed to pointer~~
>
> ~~A &&r2 = const_cast<A&&>(CA{});   // OK~~
> ~~```~~
>
> ~~-- end example]~~

4. Change in 7.6.1.11 [expr.const.cast] paragraph 4 as follows:

> For two object types T1 and T2, if a pointer to T1 can be explicitly converted to the type "pointer to T2" using a const_cast, then the following conversions can also be made:
> - an lvalue of type T1 can be explicitly converted to an lvalue of type T2 using the cast const_cast<T2&>;
> - a glvalue of type T1 can be explicitly converted to an xvalue of type T2 using the cast const_cast<T2&&>; and
> - if T1 is a class **or array** type, a prvalue of type T1 can be explicitly converted to an xvalue of type T2 using the cast const_cast<T2&&>. **The temporary materialization conversion is performed on v**.
>
> The result ~~of a reference const_cast~~ refers to the ~~original~~ **same** object ~~if~~ **as** the **(possibly converted)** operand ~~is a glvalue and to the result of applying the temporary materialization conversion (7.3.5 [conv.rval]) otherwise~~.
>
> [Example 2:
>
> ```
> typedef int *A[3];                  // array of 3 pointer to int
> typedef const int *const CA[3];     // array of 3 const pointer to const int
>
> auto &&r2 = const_cast<A&&>(CA{}); // OK, temporary materialization conversion is performed
> ```
>
> -- end example]

5. Remove 7.6.1.11 [expr.const.cast] paragraph 5:

> ~~A null pointer value (6.8.4 [basic.compound]) is converted to the null pointer value of the destination type. The null member pointer value (7.3.13 [conv.mem]) is converted to the null member pointer value of the destination type.~~

6. Change in 9.4.4 [dcl.init.ref] bullet 5.3 as follows:

> [Example 5: ...
>
> ```
> constexpr int f() {
>   const int &x = 42;
>   const_cast<int &>(x) = 1;  // undefined behavior
>   return x;
> }
> constexpr int z = f();          // error: not a constant expression
> ```
>
> ```
> typedef int *AP[3];          // array of 3 pointer to int
> typedef const int *const ACPC[3]; // array of 3 const pointer to const int
> ACPC &&r = AP{};             // binds directly
> ```

-- end example]

This resolution also resolve issue 1965.

2890. Defining members of local classes

Section: 11.6 [class.local] **Status:** ready **Submitter:** Brian Bi **Date:** 2024-03-08

Subclause 11.6 [class.local] paragraph 3 establishes restrictions on the definition of classes nested within local classes, but it is unclear which restrictions exist for other members of local classes.

Possible resolution [SUPERSEDED]:

Change in 11.6 [class.local] paragraph 3 as follows:

> ~~If class X is a local class, a nested class Y may be declared in class X and later defined in the definition of class X or be later defined in the same scope as the definition of class X.~~ A class nested within a local class is a local class. **A member of a local class X shall be declared only in the definition of X or the nearest enclosing block scope of X.**

CWG 2024-06-14

The implementation status quo is that no members of local classes other than nested classes can be defined at block scope.

Proposed resolution (approved by CWG 2024-11-08):

Change in 11.6 [class.local] paragraph 3 as follows:

> ~~If class X is a local class, a nested class Y may be declared in class X and later defined in the definition of class X or be later defined in the same scope as the definition of class X.~~ A class nested within a local class is a local class. **A member of a local class X shall be declared only in the definition of X or, if the member is a nested class, in the nearest enclosing block scope of X.**

2894. Functional casts create prvalues of reference type

Section: 7.6.1.4 [expr.type.conv] **Status:** ready **Submitter:** Jan Schultke **Date:** 2024-05-14

(From submission #536.)

For `T{...}`, the rule in 7.6.1.4 [expr.type.conv] paragraph 2 yields a prvalue of reference type if `T` is a reference type:

> ... Otherwise, the expression is a prvalue of the specified type whose result object is direct-initialized (9.4) with the initializer. ...

Also, it should be clarified that `void(1, 2)` and `void{1}` are ill-formed.

Possible resolution [SUPERSEDED]:

Change in 7.6.1.4 [expr.type.conv] paragraph 1 and 2 as follows:

> A *simple-type-specifier* (9.2.9.3 [dcl.type.simple]) or *typename-specifier* (13.8 [temp.res]) followed by a parenthesized optional *expression-list* or by a *braced-init-list* (the initializer) constructs a value of the specified type given the initializer. If the type is a placeholder for a deduced class type, it is replaced by the return type of the function selected by overload resolution for class template deduction (12.2.2.9 [over.match.class.deduct]) for the remainder of this subclause. Otherwise, if the type contains a placeholder type, it is replaced by the type determined by placeholder type deduction (9.2.9.7.2 [dcl.type.auto.deduct]). **Let `T` denote the resulting type.** [Example: ...]
>
> If the initializer is a parenthesized single expression, the type conversion expression is equivalent to the corresponding cast expression (7.6.3 [expr.cast]). Otherwise, if ~~the type~~ **`T`** is *cv* `void` ~~and~~ **,** the initializer ~~is~~ **shall be** `()` or `{}` (after pack expansion, if any), **and** the expression is a prvalue of type void that performs no initialization. Otherwise, the expression ~~is a prvalue of the specified type whose result object is direct-initialized (9.4 [dcl.init]) with the initializer~~ **has the same effect as direct-initializing an invented variable `t` of type `T` from the initializer and then using `t` as the result of the expression. The result is an lvalue if `T` is an lvalue reference type or an rvalue reference to function type, an xvalue if `T` is an rvalue reference to object type, and a prvalue otherwise**. If the initializer is a parenthesized optional *expression-list*, ~~the specified type~~ **`T`** shall not be an array type.

CWG 2024-06-14

The resolution above introduces an additional variable even for a prvalue, which defeats mandatory copy elision. Use the pattern from 7.6.1.9 [expr.static.cast] paragraph 4 instead.

Proposed resolution (approved by CWG 2024-10-25):

Change in 7.6.1.4 [expr.type.conv] paragraph 1 and 2 as follows, move the example to the bottom, and add bullets:

> A *simple-type-specifier* (9.2.9.3 [dcl.type.simple]) or *typename-specifier* (13.8 [temp.res]) followed by a parenthesized optional *expression-list* or by a *braced-init-list* (the initializer) constructs a value of the specified type given the initializer. If the type is a placeholder for a deduced class type, it is replaced by the return type of the function selected by overload resolution for class template deduction (12.2.2.9 [over.match.class.deduct]) for the remainder of this subclause. Otherwise, if the type contains a placeholder type, it is replaced by the type determined by placeholder type deduction (9.2.9.7.2 [dcl.type.auto.deduct]). **Let `T` denote the resulting type.** ~~[Example: ...]~~

Then:

- If the initializer is a parenthesized single expression, the type conversion expression is equivalent to the corresponding cast expression (7.6.3 [expr.cast]).
- Otherwise, if ~~the type~~ **`T`** is *cv* `void` ~~and~~ **,** the initializer ~~is~~ **shall be** `()` or `{}` (after pack expansion, if any), **and** the expression is a prvalue of type void that performs no initialization.
- **Otherwise, if `T` is a reference type, the expression has the same effect as direct-initializing an invented variable `t` of type `T` from the initializer and then using `t` as the result of the expression; the result is an lvalue if `T` is an lvalue reference type or an rvalue reference to function type and an xvalue otherwise.**
- Otherwise, the expression is a prvalue of ~~the specified~~ type **`T`** whose result object is direct-initialized (9.4 [dcl.init]) with the initializer.

If the initializer is a parenthesized optional *expression-list*, ~~the specified type~~ **`T`** shall not be an array type.

[Example: ...]

2899. Bad value representations should cause undefined behavior

Section: 7.3.2 [conv.lval] **Status:** ready **Submitter:** Jan Schultke **Date:** 2024-06-05

(From editorial issue #7051.)

Consider:

```
static_assert(sizeof(bool) == 1);  // assumption for the example
bool f() {
  char c = 2;
  bool b = true;
  memcpy(&b, &c, 1);     // #1
  return b;              // #2
}
```

Assuming that false and true are represented as 0 and 1, the value representation of `b` now has a bad value. This should, but does not, result in undefined behavior during the lvalue-to-rvalue conversion at #2.

Proposed resolution (approved by CWG 2024-08-16):

Change in 7.3.2 [conv.lval] paragraph 3 as follows:

> The result of the conversion is determined according to the following rules:
>
> - If T is cv std::nullptr_t, the result is a null pointer constant (7.3.12 [conv.ptr]). [*Note 1:* Since the conversion does not access the object to which the glvalue refers, there is no side effect even if T is volatile-qualified (6.9.1 [intro.execution]), and the glvalue can refer to an inactive member of a union (11.5 [class.union]). —*end note*]
> - Otherwise, if T has a class type, the conversion copy-initializes the result object from the glvalue.
> - Otherwise, if the object to which the glvalue refers contains an invalid pointer value (6.7.5.5.3 [basic.stc.dynamic.deallocation]), the behavior is implementation-defined.
> - Otherwise, if the bits in the value representation of the object to which the glvalue refers are not valid for the object's type, the behavior is undefined. [Example:
>
>   ```
>   bool f() {
>     bool b = true;
>     char c = 42;
>     memcpy(&b, &c, 1);
>     return b;           // undefined behavior if 42 is not a valid value representation for bool
>   }
>   ```
>
>   -- end example]
> - Otherwise, the object indicated by the glvalue is read (3.1 [defns.access]), and the value contained in the object is the prvalue result. ~~If the result is an erroneous value (6.7.4 [basic.indet]) and the bits in the value representation are not valid for the object's type, the behavior is undefined.~~

2901. Unclear semantics for near-match aliased access

Section: 7.2.1 [basic.lval] **Status:** ready **Submitter:** Jan Schultke **Date:** 2024-06-14

(From submission #548.)

Subclause 7.2.1 [basic.lval] paragraph 11 specifies:

> ... If a program attempts to access (3.1 [defns.access]) the stored value of an object through a glvalue through which it is not type-accessible, the behavior is undefined. ...

Thus, access (read or write) to an `int` object using an lvalue of type `unsigned int` is valid. However, 7.3.2 [conv.lval] bullet 3.4 does not specify the resulting value when, for example, the object contains the value -1:

- Otherwise, the object indicated by the glvalue is read (3.1 [defns.access]), and the value contained in the object is the prvalue result. ...

Similarly, 7.6.19 [expr.ass] paragraph 2 is silent for the assignment case:

> In simple assignment (=), the object referred to by the left operand is modified (3.1 [defns.access]) by replacing its value with the result of the right operand.

Any concerns about accesses to the object representation are handled in the context of P1839.

Proposed resolution (approved by CWG 2024-09-13):

1. Change in 7.3.2 [conv.lval] bullet 3.4 as follows:

   - Otherwise, the object indicated by the glvalue is read (3.1 [defns.access]) ~~, and the value contained in the object is the prvalue result~~. **Let *V* be the value contained in the object. If `T` is an integer type, the prvalue result is the value of type `T` congruent (6.8.2 [basic.fundamental]) to *V*, and *V* otherwise.** ...

2. Change in 7.6.1.6 [expr.post.incr] paragraph 1 as follows:

   The value of a postfix ++ expression is the value ~~of~~ **obtained by applying the lvalue-to-rvalue conversion (7.3.2 [conv.lval]) to** its operand. [*Note 1:* The value obtained is a copy of the original value. —*end note*] ...

3. Change in 7.6.19 [expr.ass] paragraph 2 as follows:

   In simple assignment (=), **let *V* be the result of the right operand;** the object referred to by the left operand is modified (3.1 [defns.access]) by replacing its value with ~~the result of the right operand~~ **V or, if the object is of integer type, with the value congruent (6.8.2 [basic.fundamental]) to V**.

2905. Value-dependence of *noexcept-expression*

Section: 13.8.3.4 [temp.dep.constexpr] **Status:** ready **Submitter:** Mital Ashok **Date:** 2024-06-16

(From submission #554.)

The following examples of *noexcept-expression*s are not specified to be value-dependent, but ought to be, because value-initialization of `T` might throw an exception.

```
template<typename T>
void f() {
  noexcept(static_cast<int>(T{}));
  noexcept(typeid(*T{}));
  noexcept(delete T{});
}
```

Proposed resolution (approved by CWG 2024-11-19):

1. Change in 13.8.3.4 [temp.dep.constexpr] paragraph 2 as follows:

   Expressions of the following form are value-dependent if the *unary-expression* or expression is *type-dependent* or the *type-id* is dependent:

   ```
   sizeof unary-expression
   sizeof ( type-id )
   typeid ( expression )
   typeid ( type-id )
   alignof ( type-id )
   ~~noexcept ( expression )~~
   ```

2. Change in 13.8.3.4 [temp.dep.constexpr] paragraph 3 as follows:

   Expressions of the following form are value-dependent if either the *type-id* ~~or~~ **,** *simple-type-specifier***, or *typename-specifier*** is dependent or the expression or *cast-expression* is value-dependent **or any *expression* in the *expression-list* is value-dependent or any *assignment-expression* in the *braced-init-list* is value-dependent**:

   ```
   simple-type-specifier ( expression-list_opt )
   typename-specifier ( expression-list_opt )
   simple-type-specifier braced-init-list
   typename-specifier braced-init-list
   static_cast < type-id > ( expression )
   const_cast < type-id > ( expression )
   reinterpret_cast < type-id > ( expression )
   dynamic_cast < type-id > ( expression )
   ( type-id ) cast-expression
   ```

3. Add a new paragraph before 13.8.3.4 [temp.dep.constexpr] paragraph 5 as follows:

   A *noexcept-expression* (7.6.2.7 [expr.unary.noexcept]) is value-dependent if its *expression* involves a template parameter.

   An expression of the form &*qualified-id* where ...

2906. Lvalue-to-rvalue conversion of class types for conditional operator

Section: 7.6.16 [expr.cond] **Status:** ready **Submitter:** Jan Schultke **Date:** 2024-06-08

(From submission #550.)

There are two known situations where the lvalue-to-rvalue conversion is applied to class types, which can be non-constexpr even if the resulting copy constructor invocation would be constexpr (7.7 [expr.const] bullet 5.9). The other such situation is 7.6.1.3 [expr.call] paragraph 11. Here, the concern is with 7.6.16 [expr.cond] paragraph 7, which can be invoked for class types; for example:

```
struct S {};
S a;
constexpr S b = a;          // OK, call to implicitly-declared copy constructor
constexpr S d = false ? S{} : a; // error: lvalue-to-rvalue conversion of 'a' is not a constant expression
```

Major implementations disagree with the ill-formed outcome.

Proposed resolution (approved by CWG 2024-06-26):

Change in 7.6.16 [expr.cond] paragraph 7 as follows:

> Otherwise, the result is a prvalue. If the second and third operands do not have the same type, ...
>
> ~~Lvalue-to-rvalue (7.3.2 [conv.lval]), array-to-pointer~~ **Array-to-pointer** (7.3.3 [conv.array])~~,~~ and function-to-pointer (7.3.4 [conv.func]) standard conversions are performed on the second and third operands. After those conversions, one of the following shall hold:
>
> - The second and third operands have the same type; the result is of that type and the result ~~object~~ is ~~initialized~~ **copy-initialized** using the selected operand.
> - The second and third operands have arithmetic or enumeration type; the usual arithmetic conversions (7.4 [expr.arith.conv]) are performed to bring them to a common type, and the result is of that type.
> - One or both of the second and third operands have pointer type; **lvalue-to-rvalue (7.3.2 [conv.lval]),** pointer ~~conversions~~ (7.3.12 [conv.ptr]), function pointer ~~conversions~~ (7.3.14 [conv.fctptr]), and qualification conversions (7.3.6 [conv.qual]) are performed to bring them to their composite pointer type (7.2.2 [expr.type]). The result is of the composite pointer type.
> - One or both of the second and third operands have pointer-to-member type; **lvalue-to-rvalue (7.3.2 [conv.lval]),** pointer to member ~~conversions~~ (7.3.13 [conv.mem]), function pointer ~~conversions~~ (7.3.14 [conv.fctptr]), and qualification conversions (7.3.6 [conv.qual]) are performed to bring them to their composite pointer type (7.2.2 [expr.type]). The result is of the composite pointer type.
> - Both the second and third operands have type std::nullptr_t or one has that type and the other is a null pointer constant. The result is of type std::nullptr_t.

2907. Constant lvalue-to-rvalue conversion on uninitialized `std::nullptr_t`

Section: 7.7 [expr.const] **Status:** ready **Submitter:** Jim X **Date:** 2023-01-10

(From submission #215.)

Consider:

```
void f() {
  std::nullptr_t np;        // uninitialized, thus np contains an erroneous value
  constexpr void *p1 = np; // error: converted initializer is not a constant expression
}
```

The lvalue-to-rvalue conversion on `np` does not actually read the value of `np` (7.3.2 [conv.lval] bullet 3.1), yet the situation is made ill-formed by 7.7 [expr.const] bullet 5.9.

Proposed resolution (approved by CWG 2024-10-11):

Change in 7.7 [expr.const] bullet 5.9 as follows:

> An expression E is a core constant expression unless the evaluation of E, following the rules of the abstract machine (6.9.1 [intro.execution]), would evaluate one of the following:
>
> - ...
> - an lvalue-to-rvalue conversion (7.3.2 [conv.lval]) unless it is applied to
>   - a glvalue of type *cv* `std::nullptr_t`,
>   - a non-volatile glvalue that refers to an object that is usable in constant expressions, or
>   - a non-volatile glvalue of literal type that refers to a non-volatile object whose lifetime began within the evaluation of E;
> - ...

2908. Counting physical source lines for `__LINE__`

Section: 15.7 [cpp.line] **Status:** ready **Submitter:** Alisdair Meredith **Date:** 2024-06-17

Given the existing implementation divergence, it should be clarified that `__LINE__` counts physical source lines, not logical ones.

Proposed resolution (approved by CWG 2024-08-16):

Change in 15.7 [cpp.line] paragraph 2 as follows:

> The *line number* of the current source line is **the line number of the current physical source line, i.e. it is** one greater than the number of new-line characters read or introduced in translation phase 1 (5.2 [lex.phases]) while processing the source file to the current token.

2909. Subtle difference between constant-initialized and constexpr

Section: 7.7 [expr.const] **Status:** ready **Submitter:** CWG **Date:** 2024-06-24

Subclause 7.7 [expr.const] paragraph 2 defines "constant-initialized" using the following rule:

> A variable or temporary object o is *constant-initialized* if
>
> - either it has an initializer or its default-initialization results in some initialization being performed, and
> - ...

However, the rules for `constexpr` are slightly different, per 9.2.6 [dcl.constexpr] paragraph 6:

> A constexpr specifier used in an object declaration declares the object as const. Such an object shall have literal type and shall be initialized. ...

The difference manifests for an example such as:

```
struct S {};
int main() {
  constexpr S s;       // OK
  constexpr S s2 = s;  // error: s is not constant-initialized
}
```

Is the difference intentional?

Proposed resolution (approved by CWG 2024-10-25):

Change in 7.7 [expr.const] paragraph 2 as follows:

> A variable or temporary object o is constant-initialized if
>
> - either it has an initializer or its ~~default-initialization results in some initialization being performed~~ **type is const-default-constructible (9.4.1 [dcl.init.general])**, and
> - ...

2910. Effect of *requirement-parameter-list*s on odr-usability

Section: 6.3 [basic.def.odr] **Status:** ready **Submitter:** Hubert Tong **Date:** 2024-06-24

(From submission #561.)

Consider:

```
bool f() {
  constexpr int z = 42;
  return requires {
    sizeof(int [*&z]);
  } && requires (int x) {
    sizeof(int [*&z]);
  };
}
```

The second *requires-expression* introduces a function parameter scope according to 6.4.4 [basic.scope.param]. This affects odr-usability as specified in 6.3 [basic.def.odr] paragraph 10, but the two *requires-expression* in the example ought to actually behave the same.

Proposed resolution (approved by CWG 2024-11-19):

Change in 6.3 [basic.def.odr] bullet 10.2.2 as follows:

> A local entity (6.1 [basic.pre]) is odr-usable in a scope (6.4.1 [basic.scope.scope]) if:
>
> - ...
> - for each intervening scope (6.4.1 [basic.scope.scope]) between the point at which the entity is introduced and the scope (where *this is considered to be introduced within the innermost enclosing class or non-lambda function definition scope), either:
>   - the intervening scope is a block scope, or
>   - the intervening scope is the function parameter scope of a *lambda-expression* **or *requires-expression***, or
>   - the intervening scope is the lambda scope of a *lambda-expression* that has a *simple-capture* naming the entity or has a *capture-default*, and the block scope of the *lambda-expression* is also an intervening scope.

2911. Unclear meaning of expressions "appearing within" subexpressions

Section: 7.5.8.1 [expr.prim.req.general] **Status:** ready **Submitter:** Hubert Tong **Date:** 2024-06-24

(From submission #562.)

Subclause 7.5.8.1 [expr.prim.req.general] paragraph 2 specifies:

> A *requires-expression* is a prvalue of type bool whose value is described below. Expressions appearing within a *requirement-body* are unevaluated operands (7.2.3 [expr.context]).

A *constant-expression* used as a non-type template argument "appearing within" the *requirement-body* should not be considered an "unevaluated operand". Similarly, bodies of *lambda-expression*s should not be in focus of "appearing within".

Proposed resolution (approved by CWG 2024-10-11):

1. Change in 7.2.3 [expr.context] paragraph 1 as follows:

> In some contexts, unevaluated operands appear (~~7.5.8 [expr.prim.req]~~ **7.5.8.2 [expr.prim.req.simple], 7.5.8.4 [expr.prim.req.compound]**, 7.6.1.8 [expr.typeid], 7.6.2.5 [expr.sizeof], 7.6.2.7 [expr.unary.noexcept], 9.2.9.6 [dcl.type.decltype], 13.1 [temp.pre], 13.7.9 [temp.concept]). An unevaluated operand is not evaluated.

2. Change in 7.5.8.1 [expr.prim.req.general] paragraph 2 as follows:

> A *requires-expression* is a prvalue of type bool whose value is described below. ~~Expressions appearing within a *requirement-body* are unevaluated operands (7.2.3 [expr.context]).~~

3. Change in 7.5.8.2 [expr.prim.req.simple] paragraph 1 as follows:

> A *simple-requirement* asserts the validity of an expression. **The *expression* is an unevaluated operand.** [*Note 1:* The enclosing *requires-expression* will evaluate to false if substitution of template arguments into the expression fails. ~~The expression is an unevaluated operand (7.2.3 [expr.context]).~~ —*end note*] ...

4. Change in 7.5.8.4 [expr.prim.req.compound] paragraph 1 as follows:

> A *compound-requirement* asserts properties of the expression E. **The *expression* is an unevaluated operand.** Substitution of template arguments (if any) and verification of semantic properties proceed in the following order:

2913. Grammar for *deduction-guide* has *requires-clause* in the wrong position

Section: 13.7.2.3 [temp.deduct.guide] **Status:** ready **Submitter:** Richard Smith **Date:** 2024-07-17

(From submission #576.)

Issue 2707 added the missing *requires-clause* to the grammar for *deduction-guide*, but the position is inconsistent with that of function declarations.

Proposed resolution (approved by CWG 2024-08-16):

Change in 13.7.2.3 [temp.deduct.guide] paragraph 1 as follows:

deduction-guide :
    explicit-specifier$_{opt}$ *template-name* (*parameter-declaration-clause*) ~~*requires-clause*$_{opt}$~~ -> *simple-template-id* **requires-clause**$_{opt}$;

2915. Explicit object parameters of type `void`

Section: 9.3.4.6 [dcl.fct] **Status:** ready **Submitter:** Anoop Rana **Date:** 2024-07-21

(From submission #578.)

Consider:

```
struct A {
  void f(this void);
};
```

This ought to be an ill-formed parameter of type `void`.

Proposed resolution (approved by CWG 2024-08-16):

Change in 9.3.4.6 [dcl.fct] paragraph 3 as follows:

> If the *parameter-declaration-clause* is empty, the function takes no arguments. A parameter list consisting of a single unnamed **non-object** parameter of non-dependent type `void` is equivalent to an empty parameter list. Except for this special case, a parameter shall not have type *cv* `void`.

2918. Consideration of constraints for address of overloaded function

Section: 12.3 [over.over] **Status:** ready **Submitter:** Richard Smith **Date:** 2024-06-26

Consider:

```
template<bool B> struct X {
  void f(short) requires B;
  void f(long);
  template<typename> void g(short) requires B;
  template<typename> void g(long);
 };
 void test(X<true> x) {
  x.f(0);           // #1, ambiguous
  x.g<int>(0);      // #2, ambiguous
  &X<true>::f;      // #3, OK!
  &X<true>::g<int>; // #4, ambiguous
}
```

For the function call cases, 12.2.4.1 [over.match.best.general] bullet 2.6 and 13.7.7.3 [temp.func.order] paragraph 6 specify that constraints are only considered if the competing overloads are otherwise basically the same. There is no corresponding restriction when taking the address of a function.

For a second issue, the treatment of placeholder type deduction is unclear:

```
template<bool B> struct X {
  void f(short) requires B;
  void f(short);
  template<typename> void g(short) requires B;
  template<typename> void g(short);
};
void test(X<true> x) {
  auto r = &X<true>::f;       // #5
  auto s = &X<true>::g<int>;  // #6
}
```

When the address of the overload set is resolved, there is a target, but the target type is `auto`, which is not properly handled.

See also issue 2572.

Proposed resolution (September, 2024) [SUPERSEDED]:

1. Change in 12.2.4.1 [over.match.best.general] bullet 2.6 as follows:

   - F1 and F2 are non-template functions and **F1 is more partial-ordering-constrained than F2 (13.5.5 [temp.constr.order])**
     - ~~they have the same non-object-parameter-type-lists (9.3.4.6 [dcl.fct]), and~~
     - ~~if they are member functions, both are direct members of the same class, and~~
     - ~~if both are non-static member functions, they have the same types for their object parameters, and~~
     - ~~F1 is more constrained than F2 according to the partial ordering of constraints described in 13.5.5 [temp.constr.order]~~,

     or if not that,

2. Change in 12.3 [over.over] paragraph 1 as follows:

   ... The target can be
   - an object or reference being initialized (9.4 [dcl.init], 9.4.4 [dcl.init.ref], 9.4.5 [dcl.init.list]),
   - the left side of an assignment (7.6.19 [expr.ass]),
   - a parameter of a function (7.6.1.3 [expr.call]),
   - a parameter of a user-defined operator (12.4 [over.oper]),
   - the return value of a function, operator function, or conversion (8.7.4 [stmt.return]),
   - an explicit type conversion (7.6.1.4 [expr.type.conv], 7.6.1.9 [expr.static.cast], 7.6.3 [expr.cast]), or
   - a non-type *template-parameter* (13.4.3 [temp.arg.nontype]).

   If the target type contains a placeholder type, placeholder type deduction is performed (9.2.9.7.2 [dcl.type.auto.deduct]), and the remainder of this subclause uses the target type so deduced.

3. Change in 12.3 [over.over] paragraph 5 as follows:

   All functions with associated constraints that are not satisfied (13.5.3 [temp.constr.decl]) are eliminated from the set of selected functions. If more than one function in the set remains, all function template specializations in the set are eliminated if the set also contains a function that is not a function template specialization. Any given non-template function F0 is eliminated if the set contains a second non-template function that is more ~~constrained~~ **partial-ordering-constrained** than F0 ~~according to the partial ordering rules of~~ **(**13.5.5 [temp.constr.order]**)**. Any given function template specialization F1 is eliminated if the set contains a second function template specialization whose function template is more specialized than the function template of F1 according to the partial ordering rules of 13.7.7.3 [temp.func.order]. After such eliminations, if any, there shall remain exactly one selected function.

4. Split and change in 12.3 [over.over] paragraph 6 as follows:

   [Example 1: ...

   The initialization of pfe is ill-formed because no f() with type int(...) has been declared, and not because of any ambiguity. ~~For another example,~~ ~~--~~ **end example]**

[Example:

...

-- end example]

[Example:

```
template<bool B> struct X {
  void f(short) requires B;
  void f(long);
  template<typename> void g(short) requires B;
  template<typename> void g(long);
};
void test() {
  &X<true>::f;       // error: ambiguous; constraints are not considered
  &X<true>::g<int>; // error: ambiguous; constraints are not considered
}
```

-- end example]

5. Add a paragraph at the end of 13.5.5 [temp.constr.order]:

A declaration D1 is more constrained than another declaration D2 when D1 is at least as constrained as D2, and D2 is not at least as constrained as D1. [Example: ...]

A non-template function F1 is more *partial-ordering-constrained* than a non-template function F2 if

- they have the same non-object-parameter-type-lists (9.3.4.6 [dcl.fct], and
- if they are member functions, both are direct members of the same class, and
- if both are non-static member functions, they have the same types for their object parameters, and
- the declaration of F1 is more constrained than the declaration of F2.

6. Change in 13.10.3.2 [temp.deduct.call] paragraph 6 as follows:

When P is a function type, function pointer type, or pointer-to-member-function type:

- If the argument is an overload set containing one or more function templates, the parameter is treated as a non-deduced context.
- If the argument is an overload set (not containing function templates), trial argument deduction is attempted using each of the members of the set. ~~If deduction succeeds for only one of the overload set members, that member is used as the argument value for the deduction. If deduction succeeds for more than one member of the overload set~~ **If all successful deductions yield the same deduced A, that deduced A is the result of deduction; otherwise,** the parameter is treated as a non-deduced context.

7. Add a new paragraph at the end of 13.10.3.2 [temp.deduct.call] as follows:

[Example:

```
// All arguments for placeholder type deduction (9.2.9.7.2 [dcl.type.auto.deduct]) yield the same deduced type.
template<bool B> struct X {
  void f(short) requires B;   // #1
  void f(short);              // #2
};
void test() {
  auto x = &X<true>::f;       // OK, deduces void(*)(short), selects #1
  auto y = &X<false>::f;       // OK, deduces void(*)(short), selects #2
}
```

-- end example]

8. Change in 13.10.3.6 [temp.deduct.type] bullet 5.6 as follows:

- A function parameter for which the associated argument is an overload set ~~(12.3 [over.over])~~, and one or more of the following apply:
  - ~~more than one function matches~~ **functions that do not all have the same function type match** the function parameter type (resulting in an ambiguous deduction), or
  - no function matches the function parameter type, or
  - the overload set supplied as an argument contains one or more function templates.

9. Add a section to C.1.4 [diff.cpp23.temp]:

Affected subclause: 13.10.3.2 [temp.deduct.call]
Change: Template argument deduction from overload sets succeeds in more cases.
Rationale: Allow consideration of constraints to disambiguate overload sets used as parameters in function calls.
Effect on original feature: Valid C++ 2023 code may become ill-formed.
[Example 1:

```
template <typename T>
void f(T &&, void (*)(T &&));

void g(int &);
inline namespace A {
  void g(short &&);
}
inline namespace B {
  void g(short &&);
}

void q() {
```

```
  int x;
  f(x, g);          // ill-formed; previously well-formed, deducing T=int&
}
```

-- end example]

CWG 2024-09-27

Functions whose constraints are not satisfied should be excluded from the overload set before attempting deduction. Also, clarify that 12.3 [over.over] applies after deduction is complete.

Proposed resolution (approved by CWG 2024-11-19):

1. Change in 12.2.4.1 [over.match.best.general] bullet 2.6 as follows:

   - F1 and F2 are non-template functions and **F1 is more partial-ordering-constrained than F2 (13.5.5 [temp.constr.order])**
     - ~~they have the same non-object-parameter-type-lists (9.3.4.6 [dcl.fct]), and~~
     - ~~if they are member functions, both are direct members of the same class, and~~
     - ~~if both are non-static member functions, they have the same types for their object parameters, and~~
     - ~~F1 is more constrained than F2 according to the partial ordering of constraints described in 13.5.5 [temp.constr.order],~~

   or if not that,

2. Change in 12.3 [over.over] paragraph 1 as follows:

   ... The target can be
   - an object or reference being initialized (9.4 [dcl.init], 9.4.4 [dcl.init.ref], 9.4.5 [dcl.init.list]),
   - the left side of an assignment (7.6.19 [expr.ass]),
   - a parameter of a function (7.6.1.3 [expr.call]),
   - a parameter of a user-defined operator (12.4 [over.oper]),
   - the return value of a function, operator function, or conversion (8.7.4 [stmt.return]),
   - an explicit type conversion (7.6.1.4 [expr.type.conv], 7.6.1.9 [expr.static.cast], 7.6.3 [expr.cast]), or
   - a non-type *template-parameter* (13.4.3 [temp.arg.nontype]).

   If the target type contains a placeholder type, placeholder type deduction is performed (9.2.9.7.2 [dcl.type.auto.deduct]), and the remainder of this subclause uses the target type so deduced.

3. Change in 12.3 [over.over] paragraph 5 as follows:

   All functions with associated constraints that are not satisfied (13.5.3 [temp.constr.decl]) are eliminated from the set of selected functions. If more than one function in the set remains, all function template specializations in the set are eliminated if the set also contains a function that is not a function template specialization. Any given non-template function F0 is eliminated if the set contains a second non-template function that is more ~~constrained~~ **partial-ordering-constrained** than F0 ~~according to the partial ordering rules of~~ **(**13.5.5 [temp.constr.order]**)**. Any given function template specialization F1 is eliminated if the set contains a second function template specialization whose function template is more specialized than the function template of F1 according to the partial ordering rules of 13.7.7.3 [temp.func.order]. After such eliminations, if any, there shall remain exactly one selected function.

4. Split and change in 12.3 [over.over] paragraph 6 as follows:

   [Example 1: ...

   The initialization of pfe is ill-formed because no f() with type int(...) has been declared, and not because of any ambiguity. ~~For another example,~~ **-- end example]**

   [Example:

   ...

   -- end example]

   [Example:

```
template<bool B> struct X {
  void f(short) requires B;
  void f(long);
  template<typename> void g(short) requires B;
  template<typename> void g(long);
};
void test() {
  &X<true>::f;       // error: ambiguous; constraints are not considered
  &X<true>::g<int>; // error: ambiguous; constraints are not considered
}
```

   -- end example]

5. Add a paragraph at the end of 13.5.5 [temp.constr.order]:

   A declaration D1 is more constrained than another declaration D2 when D1 is at least as constrained as D2, and D2 is not at least as constrained as D1. [Example: ...]

   A non-template function F1 is *more partial-ordering-constrained* than a non-template function F2 if

   - they have the same non-object-parameter-type-lists (9.3.4.6 [dcl.fct], and
   - if they are member functions, both are direct members of the same class, and
   - if both are non-static member functions, they have the same types for their object parameters, and
   - the declaration of F1 is more constrained than the declaration of F2.

6. Change in 13.10.3.2 [temp.deduct.call] paragraph 6 as follows:

When P is a function type, function pointer type, or pointer-to-member-function type:

- If the argument is an overload set containing one or more function templates, the parameter is treated as a non-deduced context.
- If the argument is an overload set (not containing function templates), trial argument deduction is attempted using each of the members of the set **whose associated constraints (13.5.2 [temp.constr.constr]) are satisfied**. ~~If deduction succeeds for only one of the overload set members, that member is used as the argument value for the deduction. If deduction succeeds for more than one member of the overload set~~ **If all successful deductions yield the same deduced A, that deduced A is the result of deduction; otherwise,** the parameter is treated as a non-deduced context.

7. Add a new paragraph at the end of 13.10.3.2 [temp.deduct.call] as follows:

[Example:

```
// All arguments for placeholder type deduction (9.2.9.7.2 [dcl.type.auto.deduct]) yield the same deduced type.
template<bool B> struct X {
  static void f(short) requires B;   // #1
  static void f(short);              // #2
};
void test() {
  auto x = &X<true>::f;       // OK, deduces void(*)(short), selects #1
  auto y = &X<false>::f;       // OK, deduces void(*)(short), selects #2
}
```

-- end example]

8. Change in 13.10.3.6 [temp.deduct.type] bullet 5.6 as follows:

- A function parameter for which the associated argument is an overload set ~~(12.3 [over.over]), and~~ **such that** one or more of the following apply:
  - ~~more than one function matches~~ **functions whose associated constraints are satisfied and that do not all have the same function type match** the function parameter type (resulting in an ambiguous deduction), or
  - no function **whose associated constraints are satisfied** matches the function parameter type, or
  - the overload set supplied as an argument contains one or more function templates.

  [Note: A particular function from the overload set is selected (12.3 [over.over]) after template argument deduction has succeeded (13.10.4 [temp.over]). -- end note]

9. Add a section to C.1.4 [diff.cpp23.temp]:

Affected subclause: 13.10.3.2 [temp.deduct.call]
Change: Template argument deduction from overload sets succeeds in more cases.
Rationale: Allow consideration of constraints to disambiguate overload sets used as parameters in function calls.
Effect on original feature: Valid C++ 2023 code may become ill-formed.
[Example 1:

```
template <typename T>
void f(T &&, void (*)(T &&));

void g(int &);              // #1
inline namespace A {
  void g(short &&);         // #2
}
inline namespace B {
  void g(short &&);         // #3
}

void q() {
  int x;
  f(x, g);          // ill-formed; previously well-formed, deducing T=int&
}
```

There is no change to the applicable deduction rules for the individual `g` candidates: Type deduction from #1 does not succeed; type deductions from #2 and #3 both succeed. -- end example]

2919. Conversion function candidates for initialization of const lvalue reference

Section: 12.2.2.7 [over.match.ref] **Status:** ready **Submitter:** Brian Bi **Date:** 2024-07-18

There is implementation divergence handling the following example:

```
struct A {
  A(const A&) = delete;
};
struct B {
  operator A&&();
};
const A& r = B();
```

Conversion to an lvalue pursuant to 9.4.4 [dcl.init.ref] bullet 5.1 fails due to the attempt to invoke a deleted function, but conversion to an rvalue according to 9.4.4 [dcl.init.ref] bullet 5.3 would succeed, except that 12.2.2.7 [over.match.ref] bullet 1.1 hinges on the target type of the initialization, not the target type of the conversion.

Proposed resolution (approved by CWG 2024-08-16):

Change in 12.2.2.7 [over.match.ref] bullet 1.1 as follows:

> Let R be a set of types including
>
> - "lvalue reference to cv2 T2" (when ~~initializing~~ **converting to** an lvalue ~~reference or an rvalue reference to function~~) and
> - "cv2 T2" and "rvalue reference to cv2 T2" (when ~~initializing~~ **converting to** an rvalue ~~reference~~ or an lvalue ~~reference to~~ **of** function **type**)
>
> for any T2.

2921. Exporting redeclarations of entities not attached to a named module

Section: 10.2 [module.interface] **Status:** ready **Submitter:** Tomasz Kamiński **Date:** 2024-06-20

Consider:

```
export module mod;
extern "C++" void func();
export extern "C++" {
  void func();
}
```

Per 10.2 [module.interface] paragraph 6, this is currently ill-formed, but implementation treatment varies and there seems to be little rationale why this should be ill-formed for entities not attached to a named module.

The rule also makes the following example ill-formed, which was intended to be well-formed:

```
export module M;
namespace N { // external linkage, attached to global module, not exported
  void f();
}
namespace N { // error: exported namespace, redeclares non-exported namespace
  export void g();
}
```

The mental model here is that for an entity not attached to a named module, exportedness is not a meaningful property of that entity.

Proposed resolution (approved by CWG 2024-08-16):

Change in 10.2 [module.interface] paragraph 6 as follows:

> A redeclaration of an entity X is implicitly exported if X was introduced by an exported declaration; otherwise it shall not be exported **if it is attached to a named module**.

2922. constexpr placement-new is too permissive

Section: 7.7 [expr.const] **Status:** ready **Submitter:** Brian Bi **Date:** 2024-07-10

constexpr placement new requires (just) pointer-interconvertibility for the argument pointer, whereas `static_cast` from `void*` to `T*` requires similarity. Requiring pointer-interconvertibility would not allow to differentiate two members of some union of the same type; such differentiation is required diagnose access to inactive union members.

Proposed resolution (approved by CWG 2024-08-16):

Change in 7.7 [expr.const] bullet 5.18.2 as follows:

- the selected allocation function is a non-allocating form (17.6.3.4 [new.delete.placement]) with an allocated type T, where
  - the placement argument to the *new-expression* points to an object ~~that is pointer-interconvertible with an object of~~ **whose** type **is similar to** T **(7.3.6 [conv.qual])** or, if T is an array type, ~~with~~ **to** the first element of an object of **a** type **similar to** T, and
  - the placement argument points to storage whose duration began within the evaluation of E;

CWG 2024-11-19

The proposed resolution does not address the ambiguity with different union members of the same type, but is a good fix to increase consistency with `static_cast` regardless.

2924. Undefined behavior during constant evaluation

Section: 3.63 [defns.undefined] **Status:** ready **Submitter:** Jan Schultke **Date:** 2024-06-04

(From editorial issue #7042 and submission #595.)

Subclause 3.63 [defns.undefined] states:

> [Note 1 to entry: ... Evaluation of a constant expression (7.7 [expr.const]) never exhibits behavior explicitly specified as undefined in Clause 4 [intro] through Clause 15 [cpp]. —*end note*]

However, 7.7 [expr.const] bullet 5.8 excludes `[[noreturn]]` and `[[assume]]`; see also 7.7 [expr.const] paragraph 6.

Suggested resolution [SUPERSEDED]:

Change in 3.63 [defns.undefined] as follows:

> [Note 1 to entry: ... Evaluation of a constant expression (7.7 [expr.const]) never exhibits behavior explicitly specified as undefined in Clause 4 [intro] through Clause 15 [cpp]**, excluding 9.12 [dcl.attr]**. —*end note*]

CWG 2024-09-13

Admitting unbounded core-language undefined behavior in constant expressions is to be avoided. The quoted note is correct; the semantics of `[[noreturn]]` and `[[assume]]` need to be clarified.

Possible resolution [SUPERSEDED]:

1. Change in 9.12.3 [dcl.attr.assume] paragraph 1 as follows:

> ... The expression is not evaluated. If the converted expression would evaluate to true at the point where the assumption appears **or if the assumption is evaluated in a context that is manifestly constant-evaluated**, the assumption has no effect. Otherwise, the behavior is undefined.

2. Change in 9.12.11 [dcl.attr.noreturn] paragraph 2 as follows:

> If a function `f` is called where `f` was previously declared with the `noreturn` attribute**, the function call is evaluated in a context that is not manifestly constant-evaluated (7.7 [expr.const]),** and `f` eventually returns, the behavior is undefined.

CWG 2024-09-27

The suggested resolution is circular with the rules in 7.7 [expr.const] paragraph 6.

Possible resolution [SUPERSEDED]:

1. Change in 9.12.3 [dcl.attr.assume] paragraph 1 as follows:

> ... The expression is not evaluated. If the converted expression would evaluate to true at the point where the assumption appears, the assumption has no effect. Otherwise, **outside of an evaluation to determine whether an expression is a core constant expression (7.7 [expr.const]),** the behavior is undefined.

2. Change in 9.12.11 [dcl.attr.noreturn] paragraph 2 as follows:

> If a function `f` is called where `f` was previously declared with the `noreturn` attribute**, the function call is evaluated outside of an evaluation to determine whether an expression is a core constant expression (7.7 [expr.const]),** and `f` eventually returns, the behavior is undefined.

CWG 2024-10-11

Implementations have two options: Either a violation of an attribute causes an expressions not to be a constant expression, leading to runtime undefined behavior, or the attribute has no effect during constant evaluation.

Possible resolution [SUPERSEDED]:

1. Change in 7.7 [expr.const] paragraph 6:

> It is implementation-defined whether *E* is a core constant expression in the situations specified in 9.12.3 [dcl.attr.assume] and 9.12.11 [dcl.attr.noreturn].
>
> It is unspecified whether E is a core constant expression if E satisfies the constraints of a core constant expression, but evaluation of E would evaluate
>
> - an operation that has undefined behavior as specified in Clause 16 [library] through Clause 34 [exec]~~,~~ **or**
> - an invocation of the va_start macro (17.13.2 [cstdarg.syn])~~,~~**.**
> - ~~a call to a function that was previously declared with the noreturn attribute (9.12.11 [dcl.attr.noreturn]) and that call returns to its caller, or~~
> - ~~a statement with an assumption (9.12.3 [dcl.attr.assume]) whose converted *conditional-expression*, if evaluated where the assumption appears, would not disqualify E from being a core constant expression and would not evaluate to true. [*Note 5:* E is not disqualified from being a core constant expression if the hypothetical evaluation of the converted *conditional-expression* would disqualify E from being a core constant expression. —*end note*]~~

2. Change in 9.12.3 [dcl.attr.assume] paragraph 1 as follows:

> ... The expression is not evaluated.
> - If the converted expression would evaluate to true at the point where the assumption appears, the assumption has no effect.
> - Otherwise, if the statement with the assumption would be evaluated as part of an evaluation of an expression *E* that satisfies the constraints of a core constant expression (7.7 [expr.const]):
>   - If the converted expression, evaluated at the point where the assumption appears, would disqualify *E* from being a core constant expression, the assumption is ignored.
>   - Otherwise, it is implementation-defined whether *E* is a core constant expression; if *E* is evaluated as a core constant expression, the assumption has no effect.
> - Otherwise, the behavior is undefined.

3. Change in 9.12.11 [dcl.attr.noreturn] paragraph 2 as follows:

> If a function `f` is called where `f` was previously declared with the `noreturn` attribute and `f` eventually returns**:**
> - If the function call would be part of an evaluation of an expression *E* that satisfies the constraints of a core constant expression (7.7 [expr.const]), it is implementation-defined whether *E* is a core constant expression; if *E* is evaluated as a core constant expression, the attribute has no effect.
> - **Otherwise**, the behavior is undefined.

CWG 2024-10-25

CWG prefers an approach suggested by Richard Smith that defines a new term "runtime undefined behavior".

Proposed resolution (approved by CWG 2024-11-08):

1. Add to Clause 3 [intro.defs]:

> **constant evaluation** [defns.const.eval]
>
> evaluation that is performed as part of evaluating an expression as a core constant expression (7.7 [expr.const])
>
> **runtime-undefined behavior** [defns.undefined.runtime]
>
> behavior that is undefined except when it occurs during constant evaluation
>
> [Note 1 to entry: During constant evaluation,
>
> - it is implementation-defined whether runtime-undefined behavior results in the expression being deemed non-constant (as specified in 7.7 [expr.const]) and
> - runtime-undefined behavior has no other effect.]

2. Change in 7.7 [expr.const] bullet 5.8 as follows:

> - an operation that would have undefined or erroneous behavior as specified in Clause 4 [intro] through Clause 15 [cpp]~~, excluding 9.12.3 [dcl.attr.assume] and 9.12.11 [dcl.attr.noreturn]~~;

3. Add a paragraph after 7.7 [expr.const] paragraph 5 as follows:

> It is implementation-defined whether *E* is a core constant expression if *E* satisfies the constraints of a core constant expression, but evaluation of *E* has runtime-undefined behavior.

4. Change in 7.7 [expr.const] paragraph 6:

> It is unspecified whether E is a core constant expression if E satisfies the constraints of a core constant expression, but evaluation of E would evaluate
>
> - an operation that has undefined behavior as specified in Clause 16 [library] through Clause 34 [exec]~~,~~ **or**
> - an invocation of the va_start macro (17.13.2 [cstdarg.syn])~~,~~.
> - ~~a call to a function that was previously declared with the noreturn attribute (9.12.11 [dcl.attr.noreturn]) and that call returns to its caller, or~~
> - ~~a statement with an assumption (9.12.3 [dcl.attr.assume]) whose converted *conditional-expression*, if evaluated where the assumption appears, would not disqualify E from being a core constant expression and would not evaluate to true. [*Note 5:* E is not disqualified from being a core constant expression if the hypothetical evaluation of the converted *conditional-expression* would disqualify E from being a core constant expression. — *end note*]~~

5. Change in 9.12.3 [dcl.attr.assume] paragraph 1 as follows:

> ... If the converted expression would evaluate to true at the point where the assumption appears, the assumption has no effect. Otherwise, ~~the behavior is undefined~~ **evaluation of the assumption has runtime-undefined behavior**.

6. Change in 9.12.11 [dcl.attr.noreturn] paragraph 2 as follows:

> If a function f is ~~called~~ **invoked** where f was previously declared with the noreturn attribute and ~~f~~ **that invocation** eventually returns, the behavior is **runtime-**undefined.

2927. Unclear status of translation unit with `module` keyword

Section: 15.1 [cpp.pre] **Status:** ready **Submitter:** Chuanqi Xu **Date:** 2024-08-15

Consider:

```
using module = int;
module i;
int foo() {
  return i;
}
```

Is this a valid translation unit?

It is not a valid *module-file*, because the translation unit does not start with `module` or `export module` (15.4 [cpp.module]). It is also not a valid traditional *preprocessing-file* (15.1 [cpp.pre]), because `module i` is considered a preprocessing directive (15.1 [cpp.pre] bullet 1.3, which is never a *text-line* (15.1 [cpp.pre]

paragraph 2):

> A sequence of preprocessing tokens is only a *text-line* if it does not begin with a directive-introducing token. ...

A clarifying note would be appreciated.

Possible resolution (August, 2024):

Change in 15.1 [cpp.pre] paragraph 2 as follows:

> A sequence of preprocessing tokens is only a *text-line* if it does not begin with a directive-introducing token. **[Note: A source line starting with `module` *identifier* is always interpreted as a preprocessing directive, not as a *text-line*, even if parsing the source file as a *preprocessing-file* subsequently fails. -- end note]**
>
> A sequence of preprocessing tokens is only a *conditionally-supported-directive* if ...

CWG 2024-09-13

A source line starting with `module` *identifier* appearing in a macro argument may or may not be interpreted as a preprocessing directive (15.6.2 [cpp.subst] paragraph 13), thus the note as written is incorrect.

Proposed resolution (approved by CWG 2024-09-27):

Change in 15.1 [cpp.pre] paragraph 2 as follows:

> A sequence of preprocessing tokens is only a *text-line* if it does not begin with a directive-introducing token. **[Example:**
>
> ```
> using module = int;
> module i;       // not a text-line and not a control-line
> int foo() {
>   return i;
> }
> ```
>
> **The example is not a valid *preprocessing-file*. -- end example]**

2930. Unclear term "copy/move operation" in specification of copy elision

Section: 11.9.6 [class.copy.elision] **Status:** ready **Submitter:** Gabriel Dos Reis **Date:** 2024-08-16

The specification of copy elision in 11.9.6 [class.copy.elision] uses the undefined term "copy/move operation", even though the constructor actually selected might not be a copy or move constructor as specified in 11.4.5.3 [class.copy.ctor]. It is thus unclear whether copy elision can be applied even in the case a non-copy/move constructor is selected.

Proposed resolution (approved by CWG 2024-11-22):

Change in 11.9.6 [class.copy.elision] paragraph 1 through 3 as follows:

> When certain criteria are met, an implementation is allowed to omit the ~~copy/move construction of~~ **creation of** a class object **from a source object of the same type (ignoring cv-qualification)**, even if the **selected** constructor ~~selected for the copy/move operation~~ and/or the destructor for the object have side effects. In such cases, the implementation treats the source and target of the omitted ~~copy/move operation~~ **initialization** as simply two different ways of referring to the same object. If the first parameter of the selected constructor is an rvalue reference to the object's type, the destruction of that object occurs when the target would have been destroyed; otherwise, the destruction occurs at the later of the times when the two objects would have been destroyed without the optimization. [~~Footnote:~~ **Note:** Because only one object is destroyed instead of two, and ~~one copy/move constructor is not executed~~ **the creation of one object is omitted**, there is still one object destroyed for each one constructed. -- end ~~footnote~~ **note**] This elision of ~~copy/move operations~~ **object creation**, called *copy elision*, is permitted in the following circumstances (which may be combined to eliminate multiple copies):
>
> - in a `return` statement **(8.7.4 [stmt.return])** in a function with a class return type, when the *expression* is the name of a non-volatile object ***o*** with automatic storage duration (other than a function parameter or a variable introduced by the *exception-declaration* of a *handler* (14.4 [except.handle])) ~~with the same type (ignoring cv-qualification) as the function return type~~, the ~~copy/move operation~~ **copy-initialization of the result object** can be omitted by constructing ~~the object~~ ***o*** directly into the function call's ~~return~~ **result** object**;**
> - in a *throw-expression* (7.6.18 [expr.throw]), when the operand is the name of a non-volatile object ***o*** with automatic storage duration (other than a function ~~or catch-clause~~ parameter **or a variable introduced by the *exception-declaration* of a *handler***) that belongs to a scope that does not contain the innermost enclosing *compound-statement* associated with a *try-block* (if there is one), the ~~copy/move operation~~ **copy-initialization of the exception object** can be omitted by constructing ~~the object~~ ***o*** directly into the exception object**;**
> - in a coroutine (9.5.4 [dcl.fct.def.coroutine]), a copy of a coroutine parameter can be omitted and references to that copy replaced with references to the corresponding parameter if the meaning of the program will be unchanged except for the execution of a constructor and destructor for the parameter copy object**;**
> - when the *exception-declaration* of a handler (14.4 [except.handle]) declares an object ***o*** ~~of the same type (except for cv-qualification) as the exception object (14.2 [except.throw])~~, the ~~copy operation~~ **copy-initialization of *o*** can be omitted by treating the *exception-declaration* as an alias for the exception object if the meaning of the program will be unchanged except for the execution of constructors and destructors for the object declared by the *exception-declaration*. [*Note 1:* There cannot be a move from the exception object because it is always an lvalue. —*end note*]

2931. Restrictions on operator functions that are explicit object member functions

Section: 12.4.1 [over.oper.general] **Status:** ready **Submitter:** Barry Revzin **Date:** 2024-08-26

(From submission #600.)

With the introduction of explicit object member functions, the restrictions on operator functions became inconsistent. Subclause 12.4.1 [over.oper.general] paragraph 7 specifies:

> An operator function shall either
>
> - be a member function or
> - be a non-member function that has at least one non-object parameter whose type is a class, a reference to a class, an enumeration, or a reference to an enumeration.

Talking about non-object parameters in a bullet discussing non-member functions makes no sense. The following example ought to be prohibited, for consistency with `operator==(int, int)`:

```
struct B {
  bool operator==(this int, int);
  operator int() const;
};
```

Proposed resolution (approved by CWG 2024-11-22):

Change in 12.4.1 [over.oper.general] paragraph 7 as follows, removing the bullets:

> An operator function shall ~~either~~
>
> - ~~be a member function or~~
> - ~~be a non-member function that has~~ **have** at least one ~~non-object~~ **function** parameter **or implicit object parameter** whose type is a class, a reference to a class, an enumeration, or a reference to an enumeration.

2933. Dangling references

Section: 7.2.2 [expr.type] **Status:** ready **Submitter:** Brian Bi **Date:** 2024-09-04

Issue 453 clarified that there are no empty lvalues, and undefined behavior ensues when trying to create one. However, the wording now does not specify the behavior of dangling references, where the storage of the referenced object has gone away.

Proposed resolution (approved by CWG 2024-11-08):

1. Change in 6.8.2 [basic.fundamental] paragraph 4 as follows:

> A pointer value P is valid in the context of an evaluation E if P is **a pointer to function or** a null pointer value, or if it is a pointer to or past the end of an object O and E happens before the end of the duration of the region of storage for O. If a pointer value P is used in an evaluation E and P is not valid in the context of E, then ...

2. Change in 7.2.2 [expr.type] paragraph 1 as follows:

> If an expression initially has the type "reference to T" (9.3.4.3 [dcl.ref], 9.4.4 [dcl.init.ref]), the type is adjusted to T prior to any further analysis**; the value category of the expression is not altered**. ~~The expression designates~~ **Let *X* be** the object or function denoted by the reference~~, and the expression is an lvalue or an xvalue, depending on the expression~~. **If a pointer to *X* would be valid in the context of the evalution of the expression (6.8.2 [basic.fundamental]), the result designates *X*; otherwise, the behavior is undefined.**

2936. Local classes of templated functions should be part of the current instantiation

Section: 13.8.3.2 [temp.dep.type] **Status:** ready **Submitter:** Richard Smith **Date:** 2024-09-02

Consider:

```
template<class T>
void f()
{
  struct Y {
    using type = int;
  };
  Y::type y;  // error; missing typename
}
```

Since lookup of `Y` always finds the local class, regardless of `T`, the requirement to apply `typename` is too strict.

Proposed resolution (approved by CWG 2024-11-08):

Change in 13.8.3.2 [temp.dep.type] paragraph 1 as follows:

> A name or *template-id* refers to the current instantiation if it is

- ...
- in the definition of a nested class of a class template, the name of the nested class referenced as a member of the current instantiation, ~~or~~
- in the definition of a class template partial specialization or a member of a class template partial specialization, the name of the class template followed by a template argument list equivalent to that of the partial specialization (13.7.6 [temp.spec.partial]) enclosed in <> (or an equivalent template alias specialization)~~.~~**, or**
- in the definition of a templated function, the name of a local class (11.6 [class.local].

2937. Grammar for *preprocessing-file* has no normative effect

Section: 5.2 [lex.phases] **Status:** ready **Submitter:** Brian Bi **Date:** 2024-09-27

(From submission #615.)

Subclause 15.1 [cpp.pre] defines the grammar production *preprocessing-file*, but nothing in the standard specifies that a translation unit is ill-formed if it fails to match that grammar. Similarly, *translation-unit* has no effect.

Proposed resolution (approved by CWG 2024-11-08):

1. Change in 5.2 [lex.phases] bullet 1.4 as follows:

   - ...
   - **The source file is analyzed as a *preprocessing-file* (15.1 [cpp.pre]).** Preprocessing directives are executed, macro invocations are expanded, and _Pragma unary operator expressions are executed. A #include preprocessing directive causes the named header or source file to be processed from phase 1 through phase 4, recursively. All preprocessing directives are then deleted.
   - ...

2. Change in 5.2 [lex.phases] bullet 1.7 as follows:

   Whitespace characters separating tokens are no longer significant. Each preprocessing token is converted into a token (5.6 [lex.token]). The resulting tokens constitute a *translation unit* and are syntactically and semantically analyzed **as a *translation-unit* (6.6 [basic.link])** and translated.

2939. Do not allow `reinterpret_cast` from prvalue to rvalue reference

Section: 7.6.1.10 [expr.reinterpret.cast] **Status:** ready **Submitter:** Brian Bi **Date:** 2024-10-01

(From submission #617.)

In 7.6.1.10 [expr.reinterpret.cast] paragraph 11, there is an issue similar to the one described in issue 2879: The operand for a cast to reference type can be a "glvalue of type T1", which implies that a prvalue is also acceptable, because it is materialized per 7.2.1 [basic.lval] paragraph 7. However, no implementation accepts `reinterpret_cast<double&&>(0)`.

Suggested resolution [SUPERSEDED]:

1. Change in 7.6.1.10 [expr.reinterpret.cast] paragraph 1 as follows:

   The result of the expression reinterpret_cast<T>(v) is the result of converting the expression v to type T. If T is an lvalue reference type or an rvalue reference to function type, the result is an lvalue; if T is an rvalue reference to object type, the result is an xvalue; otherwise, the result is a prvalue and the lvalue-to-rvalue (7.3.2 [conv.lval]), array-to-pointer (7.3.3 [conv.array]), and function-to-pointer (7.3.4 [conv.func]) standard conversions are performed on the expression v. **The temporary materialization conversion (7.3.5 [conv.rval]) is not performed on v.** Conversions that can be performed explicitly using reinterpret_cast are listed below. No other conversion can be performed explicitly using reinterpret_cast.

2. Remove 7.6.1.10 [expr.reinterpret.cast] paragraph 3:

   ~~[*Note 1:* The mapping performed by reinterpret_cast might, or might not, produce a representation different from the original value. — *end note*]~~

3. Remove from 7.6.1.10 [expr.reinterpret.cast] paragraph 6 as follows:

   ... ~~[*Note 5:* See also 7.3.12 [conv.ptr] for more details of pointer conversions. — *end note*]~~

4. Change in 7.6.1.10 [expr.reinterpret.cast] paragraph 11 as follows:

   A glvalue of type T1, designating an object or function x, can be cast to the type “reference to T2” if an expression of type “pointer to T1” can be explicitly converted to the type “pointer to T2” using a reinterpret_cast. The result is that of *reinterpret_cast<T2 *>(p) where p is a pointer to x of type “pointer to T1”. **[Note:** No temporary is created, no copy is made, and no constructors (11.4.5 [class.ctor]) or conversion functions (11.4.8 [class.conv]) are called [Footnote: ...]. **-- end note]**

CWG 2024-10-11

The note in 7.6.1.10 [expr.reinterpret.cast] paragraph 3 should be kept, and the list of exceptions in 7.2.1 [basic.lval] paragraph 7 established by issue 2879 should be amended with a cross-reference to 7.6.1.10 [expr.reinterpret.cast].

Proposed resolution (approved by CWG 2024-11-22):

1. Change in 7.2.1 [basic.lval] paragraph 7 as follows:

Unless otherwise specified (**7.6.1.10 [expr.reinterpret.cast],** 7.6.1.11 [expr.const.cast]), whenever a prvalue appears as an operand of an operator that expects a glvalue for that operand, the temporary materialization conversion (7.3.5 [conv.rval]) is applied to convert the expression to an xvalue.

2. Remove from 7.6.1.10 [expr.reinterpret.cast] paragraph 6 as follows:

... ~~[*Note 5:* See also 7.3.12 [conv.ptr] for more details of pointer conversions. — *end note*]~~

3. Change in 7.6.1.10 [expr.reinterpret.cast] paragraph 11 as follows:

~~A~~ **If v is a** glvalue of type T1, designating an object or function x, **it** can be cast to the type "reference to T2" if an expression of type "pointer to T1" can be explicitly converted to the type "pointer to T2" using a reinterpret_cast. The result is that of *reinterpret_cast<T2 *>(p) where p is a pointer to x of type "pointer to T1". **[Note:** No temporary is **materialized (7.3.5 [conv.rval]) or** created, no copy is made, and no constructors (11.4.5 [class.ctor]) or conversion functions (11.4.8 [class.conv]) are called [Footnote: ...]. **-- end note]**

2944. Unsequenced *throw-expression*s

Section: 7.6.18 [expr.throw] **Status:** ready **Submitter:** Jan Schultke **Date:** 2024-10-23

(From submission #626.)

The semantics of unsequenced *throw-expression*s is unclear. For example:

```
(throw /* ... */, 0) + (throw /* ... */, 0);
```

This appears to cause two unsequenced transfers of control, which makes little sense. In contrast, a `co_await` expression is indivisible (6.9.1 [intro.execution] paragraph 11) per issue 2466.

Proposed resolution (approved by CWG 2024-11-08):

Change in 6.9.1 [intro.execution] paragraph 11 as follows, adding bullets:

When invoking a function (whether or not the function is inline), every argument expression and the postfix expression designating the called function are sequenced before every expression or statement in the body of the called function. For each

- function invocation**,**
- evaluation of an *await-expression* **(7.6.2.4 [expr.await]), or**
- evaluation of a *throw-expression* (7.6.18 [expr.throw])

F, each evaluation that does not occur within F but is evaluated on the same thread and as part of the same signal handler (if any) is either sequenced before all evaluations that occur within F or sequenced after all evaluations that occur within F; [Foonote: ...] if F invokes or resumes a coroutine (7.6.2.4 [expr.await]), only evaluations subsequent to the previous suspension (if any) and prior to the next suspension (if any) are considered to occur within F.

CWG 2024-11-08

Check with implementers (in particular MSVC) that the proposed resolution is acceptable.